



AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Fall 2025-26

Section: O

Course: COMPUTER ORGANIZATION AND ARCHITECTURE

Performance Tasks

Supervised By SHANDIPAN PAUL

Submitted By

SN	Name	ID
1	Afnan Bin Islam Nahin	22-49350-3

Date of Submission: 6.1.2026

Task for assembly_jump_questions.pdf

QUESTION 1: Uses A=30, B=10. Compare using unsigned (JA, JB) and signed (JG, JL).

The screenshot shows an assembly editor window with the following code:

```
01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msg1 DB 'Unsigned: A is greater than B$'
06 msg2 DB 0DH,0AH,'Signed: A is greater than B$'
07
08 .CODE
09 MAIN PROC
10 MOV AX, @DATA
11 MOV DS, AX
12
13 MOV AL, 30
14 MOV BL, 10
15
16
17 CMP AL, BL
18 JA UNSIGNED_GREATER
19 JB UNSIGNED_LESS
20
21 UNSIGNED_GREATER:
22 MOV AH, 09H
23 LEA DX, msg1
24 INT 21H
25
26
27 CMP AL, BL
28 JG SIGNED_GREATER
29 JL SIGNED_LESS
30
31 SIGNED_GREATER:
32 MOV AH, 09H
33 LEA DX, msg2
34 INT 21H
35
36 MOV AH, 4CH
37 INT 21H
38
39 UNSIGNED_LESS:
40 SIGNED_LESS:
41 MOV AH, 4CH
42 INT 21H
43
44 MAIN ENDP
45 END MAIN
```

The emulator window shows the output:

```
Unsigned: A is greater than B
Signed: A is greater than B
```

QUESTION 2: Uses X=200, Y=100. Compare using JAE, JBE, JGE, JLE.

The screenshot shows an assembly editor window with the following code:

```
01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 msg DB 'X is greater or equal to Y$'
05 .CODE
06 MAIN PROC
07 MOV AX, @DATA
08 MOV DS, AX
09
10 MOV AL, 200
11 MOV BL, 100
12 CMP AL, BL
13
14 JAE PRINT
15 JBE PRINT
16
17 PRINT:
18 MOV AH, 9
19 LEA DX, msg
20 INT 21H
21
22 MOV AH, 4CH
23 INT 21H
24 MAIN ENDP
25 END MAIN
```

The emulator window shows the output:

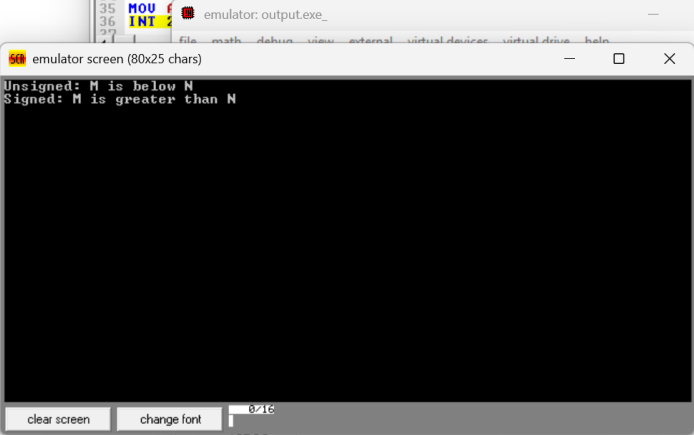
```
X is greater or equal to Y
```

QUESTION 3: Uses M=5, N=250. Compare using JB, JAE, JG, JL.

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msg1 DB 'Unsigned: M is below N$'
06 msg2 DB 0DH,0AH,'Signed: M is greater than N$'
07
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13     ; M = 5, N = 250
14     MOV AL, 5
15     MOV BL, 250
16
17     CMP AL, BL
18     JB UNSIGNED_BELOW
19     JAE UNSIGNED_ABOVE
20
21 UNSIGNED_BELOW:
22     MOV AH, 09H
23     LEA DX, msg1
24     INT 21H
25
26     CMP AL, BL
27     JG SIGNED_GREATER
28     JL SIGNED_LESS
29
30 SIGNED_GREATER:
31     MOV AH, 09H
32     LEA DX, msg2
33     INT 21H
34
35     MOV AH, 4CH
36     INT 21H
37
38 UNSIGNED_ABOVE:
39 SIGNED_LESS:
40     MOV AH, 4CH
41     INT 21H
42
43 MAIN ENDP
44 END MAIN
45
23 LEA DX, msg1
24 INT 21H
25
26 CMP AL, BL
27 JG SIGNED_GREATER
28 JL SIGNED_LESS
29
30 SIGNED_GREATER:
31 MOV AH, 09H
32 LEA DX, msg2
33 INT 21H
34
35 MOV AH, 4CH
36 INT 21H
37
38
39
40
41
42
43
44
45

```



emulator: output.exe_

emulator screen (80x25 chars)

Unsigned: M is below N
Signed: M is greater than N

clear screen change font 0/16

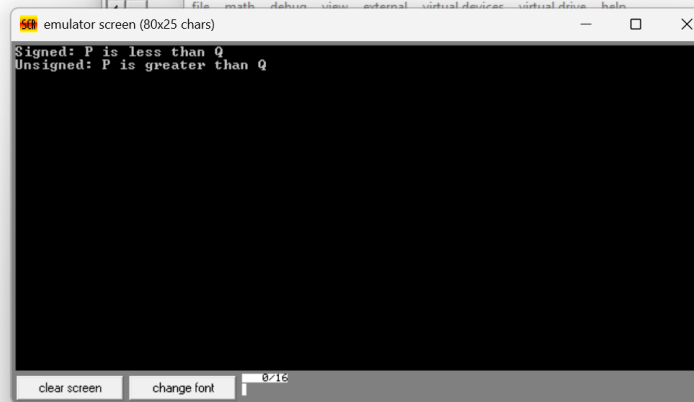
ES 0700 screen source reset aux vars debug stack flags

QUESTION 4: Uses P=-5, Q=3. Compare using JL, JG, JA, JB.

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msg1 DB 'Signed: P is less than Q$'
06 msg2 DB 0DH,0AH,'Unsigned: P is greater than Q$'
07
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13     ; P = -5, Q = 3
14     MOV AL, -5
15     MOV BL, 3
16
17     CMP AL, BL
18     JL SIGNED_LESS
19     JG SIGNED_GREATER
20
21 SIGNED_LESS:
22     MOV AH, 09H
23     LEA DX, msg1
24     INT 21H
25
26     CMP AL, BL
27     JA UNSIGNED_ABOVE
28     JB UNSIGNED_BELOW
29
30 UNSIGNED_ABOVE:
31     MOV AH, 09H
32     LEA DX, msg2
33     INT 21H
34
35     MOV AH, 4CH
36     INT 21H
37
38 SIGNED_GREATER:
39 UNSIGNED_BELOW:
40     MOV AH, 4CH
41     INT 21H
42
43 MAIN ENDP
44 END MAIN
45
23 MOV AH, 09H
24 LEA DX, msg1
25 INT 21H
26
27 CMP AL, BL
28 JA UNSIGNED_ABOVE
29 JB UNSIGNED_BELOW
30
31 UNSIGNED_ABOVE:
32 MOV AH, 09H
33 LEA DX, msg2
34 INT 21H
35
36 MOV AH, 4CH
37 INT 21H
38
39
40
41
42
43
44
45

```



emulator: output.exe_

emulator screen (80x25 chars)

Signed: P is less than Q
Unsigned: P is greater than Q

clear screen change font 0/16

ES 0700 screen source reset aux vars debug stack fla

QUESTION 5: Uses U=100, V=100. Compare using JE, JNE, JGE, JLE.

```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 msgEQ DB 'U and V are Equal$'
05 msgNE DB 'U and V are Not Equal$'
06 msgGE DB 'U is Greater than or Equal to V$'
07 msgLE DB 'U is Less than or Equal to V$'
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12     ; U = 100, V = 100
13     MOV AL, 100
14     MOV BL, 100
15     CMP AL, BL
16
17     JE EQUAL
18     JNE NOTEQUAL
19     JGE GREATER_EQUAL
20     JLE LESS_EQUAL
21
22 EQUAL:
23     MOV AH, 9
24     LEA DX, msgEQ
25     INT 21H
26     JMP EXIT
27 NOTEQUAL:
28     MOV AH, 9
29     LEA DX, msgNE
30     INT 21H
31     JMP EXIT
32
33 GREATER_EQUAL:
34     MOV AH, 9
35     LEA DX, msgGE
36     INT 21H
37     JMP EXIT
38 LESS_EQUAL:
39     MOV AH, 9
40     LEA DX, msgLE
41     INT 21H
42 EXIT:
43     MOV AH, 4CH
44     INT 21H
45 MAIN ENDP
46 END MAIN

```

```

30 INT 21H
31 JMP EXIT
32
33 GREATER_EQUAL:
34 MOV AH, 9
35 LEA DX, msgGE
36 INT 21H
37 JMP EXIT
38 LESS_EQUAL:
39 MOV AH, 9
40 LEA DX, msgLE
41 INT 21H
42 EXIT:
43 MOV AH, 9
44 INT 21H
45
46

```

emulator screen (80x25 chars)

U and V are Equal

clear screen change font 8/16

ES 0700 screen source reset aux vars debug stack

QUESTION 6: Uses D=0FFH, E=1. Compare using JA, JB, JG, JL.

```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 msgUA DB 'Unsigned: D > E$'
05 msgUB DB 'Unsigned: D < E$'
06 msgSG DB 'Signed: D > E$'
07 msgSL DB 'Signed: D < E$'
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12     ; D = 0FFH (255 unsigned, -1 signed)
13     ; E = 1
14     MOV AL, 0FFH
15     MOV BL, 1
16     CMP AL, BL
17     JA UNSIGNED_GREATER
18     JB UNSIGNED_LESS
19     JG SIGNED_GREATER
20     JL SIGNED_LESS
21
22 UNSIGNED_GREATER:
23     MOV AH, 9
24     LEA DX, msgUA
25     INT 21H
26     JMP EXIT
27 UNSIGNED_LESS:
28     MOV AH, 9
29     LEA DX, msgUB
30     INT 21H
31     JMP EXIT
32 SIGNED_GREATER:
33     MOV AH, 9
34     LEA DX, msgSG
35     INT 21H
36     JMP EXIT
37 SIGNED_LESS:
38     MOV AH, 9
39     LEA DX, msgSL
40     INT 21H
41
42 EXIT:
43     MOV AH, 4CH
44     INT 21H
45 MAIN ENDP
46 END MAIN

```

```

29 MOV AH, 9
30 INT 21H
31 JMP EXIT
32 SIGNED_GREATER:
33 MOV AH, 9
34 LEA DX, msgSG
35 INT 21H
36 JMP EXIT
37 SIGNED_LESS:
38 MOV AH, 9
39 LEA DX, msgSL
40 INT 21H
41
42 EXIT:
43 MOV AH, 9
44 INT 21H
45
46

```

emulator screen (80x25 chars)

Unsigned: D > E

clear screen change font 8/16

ES 0700 screen source reset aux vars debug stack

QUESTION 7: Uses R=-10, S=-10. Compare using JE, JNE, JG, JL

The screenshot shows an x86-64 assembler interface with the following assembly code:

```

10 .CODE
11 MAIN PROC
12     MOV AX, @DATA
13     MOV DS, AX
14
15     ; R = -10, S = -10
16     MOV AL, -10
17     MOV BL, -10
18     CMP AL, BL
19
20     JE EQUAL
21     JNE NOTEQUAL
22     JG GREATER
23     JL LESS
24
25 EQUAL:
26     MOV AH, 9
27     LEA DX, msgEQ
28     INT 21H
29     JMP EXIT
30
31 NOTEQUAL:
32     MOV AH, 9
33     LEA DX, msgNE
34     INT 21H
35     JMP EXIT
36
37 GREATER:
38     MOV AH, 9
39     LEA DX, msgGT
40     INT 21H
41     JMP EXIT
42
43 LESS:
44     MOV AH, 9
45     LEA DX, msgLT
46     INT 21H
47
48 EXIT:
49     MOV AH, 4CH
50     INT 21H
51
52 MAIN ENDP
53 END MAIN
54
36 GREATER:
37     MOV AH, 9
38     LEA DX, msgGT
39     INT 21H
40     JMP EXIT
41
42 LESS:
43     MOV AH, 9
44     LEA DX, msgLT
45     INT 21H
46
47 EXIT:
48     MOV AH, 4CH
49     INT 21H
50
51 MAIN ENDP
52 END MAIN

```

The emulator screen (80x25 chars) displays the output: "R and S are Equal".

QUESTION 8: Uses A=7, B=20, C=7. Compare using JB, JG, JL.

The screenshot shows an x86-64 assembler interface with the following assembly code:

```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     msgJB DB 'A is Below B$'
05     msgJG DB 'B is Greater than C$'
06     msgJL DB 'C is Less than B$'
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     ; A = 7, B = 20, C = 7
13     MOV AL, 7 ; A
14     MOV BL, 20 ; B
15     MOV CL, 7 ; C
16     CMP AL, BL
17     JB BELOW ; A < B
18
19     CMP BL, CL
20     JG GREATER ; B > C
21     JL LESS ; C < B
22
23 BELOW:
24     MOV AH, 9
25     LEA DX, msgJB
26     INT 21H
27     JMP EXIT
28
29 GREATER:
30     MOV AH, 9
31     LEA DX, msgJG
32     INT 21H
33     JMP EXIT
34
35 LESS:
36     MOV AH, 9
37     LEA DX, msgJL
38     INT 21H
39
40 EXIT:
41     MOV AH, 4CH
42     INT 21H
43
44 MAIN ENDP
45 END MAIN

```

The emulator screen (80x25 chars) displays the output: "A is Below B".

QUESTION 9: Uses L=128, K=2. Compare using JA, JB, JG, JL.

```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 msgUA DB 'Unsigned: L > K$'
05 msgUB DB 'Unsigned: L < K$'
06 msgSG DB 'Signed: L < K$'
07 msgSL DB 'Signed: L > K$'
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13     ; L = 128, K = 2
14     MOV AL, 128
15     MOV BL, 2
16     CMP BL, AL
17     JA  U_GREATER
18     JB  U_LESS
19     JG  S_GREATER
20     JL  S_LESS
21 U_GREATER:
22     MOV AH, 9
23     LEA DX, msgUA
24     INT 21H
25     JMP EXIT
26 U_LESS:
27     MOV AH, 9
28     LEA DX, msgUB
29     INT 21H
30     JMP EXIT
31 S_GREATER:
32     MOV AH, 9
33     LEA DX, msgSG
34     INT 21H
35     JMP EXIT
36 S_LESS:
37     MOV AH, 9
38     LEA DX, msgSL
39     INT 21H
40 EXIT:
41     MOV AH, 4CH
42     INT 21H
43 MAIN ENDP
44 END MAIN

```

emulator screen (80x25 chars)

```

Unsigned: L > K

```

QUESTION 10: Uses num1=-50, num2=200. Compare using JL, JA, JB.

```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 msgJL DB 'Signed: num1 < num2$'
05 msgJA DB 'Unsigned: num1 > num2$'
06 msgJB DB 'Unsigned: num1 < num2$'
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     ; num1 = -50, num2 = 200
13     MOV AL, -50
14     MOV BL, 200
15     CMP AL, BL
16     JL  SIGNED_LESS
17     JA  UNSIGNED_GREATER
18     JB  UNSIGNED_LESS
19
20 SIGNED_LESS:
21     MOV AH, 9
22     LEA DX, msgJL
23     INT 21H
24     JMP EXIT
25
26 UNSIGNED_GREATER:
27     MOV AH, 9
28     LEA DX, msgJA
29     INT 21H
30     JMP EXIT
31
32 UNSIGNED_LESS:
33     MOV AH, 9
34     LEA DX, msgJB
35     INT 21H
36
37 EXIT:
38     MOV AH, 4CH
39     INT 21H
40 MAIN ENDP
41 END MAIN

```

emulator screen (80x25 chars)

```

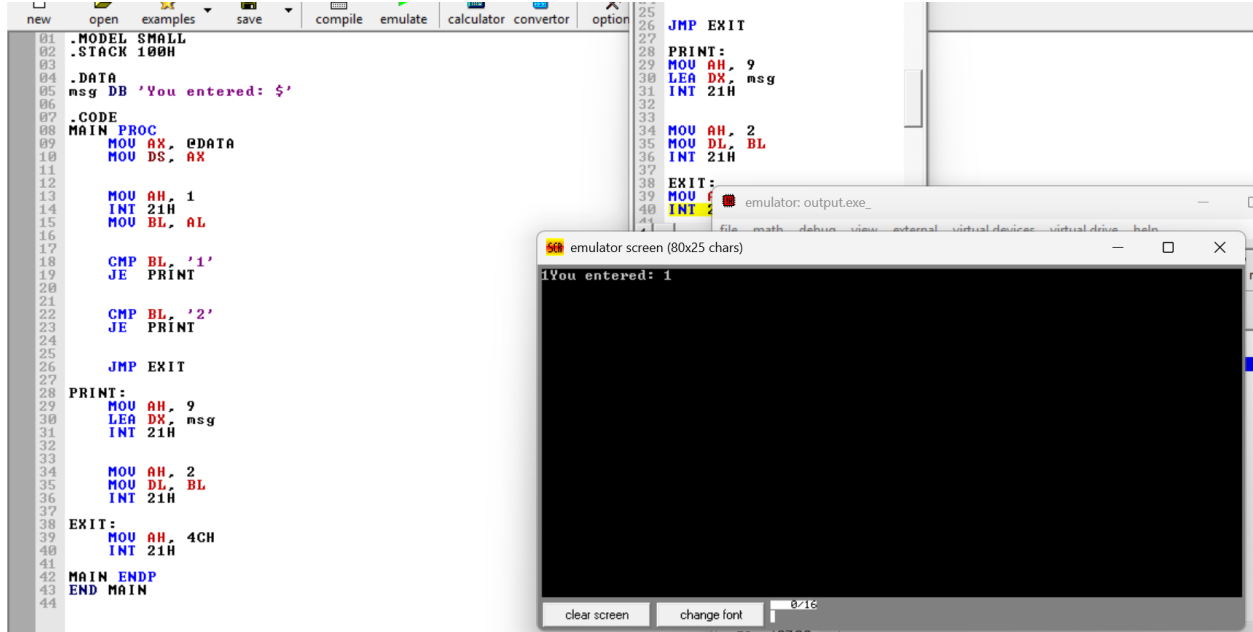
Unsigned: num1 > num2

```

Tasks for G_Today.pdf

Code Tasks:

1. Write an 8086 assembly program that reads a character and prints it only if it is '1' or '2'; otherwise terminate.

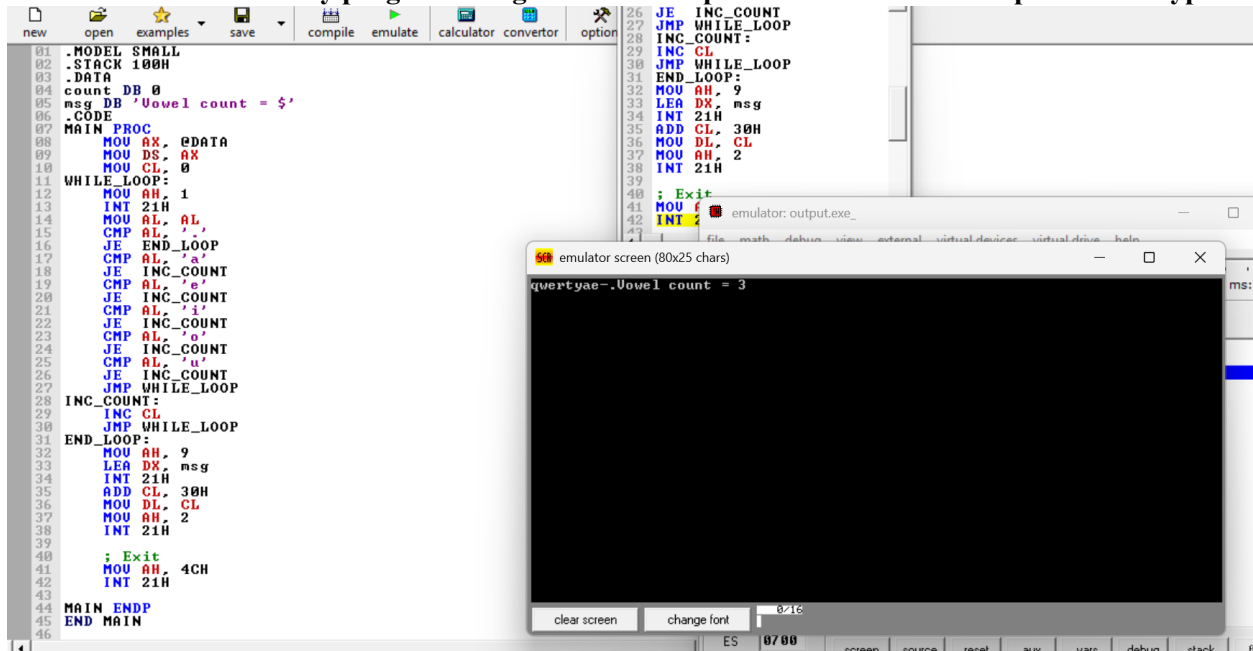


```
01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msg DB 'You entered: $'
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     MOV AH, 1
13     INT 21H
14     MOV BL, AL
15
16     CMP BL, '1'
17     JE PRINT
18
19     CMP BL, '2'
20     JE PRINT
21
22     JMP EXIT
23
24 PRINT:
25     MOV AH, 9
26     LEA DX, msg
27     INT 21H
28
29     MOV AH, 2
30     MOV DL, BL
31     INT 21H
32
33 EXIT:
34     MOV AH, 4CH
35     INT 21H
36
37 MAIN ENDP
38 END MAIN
```

emulator screen (80x25 chars)

1You entered: 1

2. Write an 8086 assembly program using a WHILE loop to count vowels until a period '.' is typed.



```
01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 count DB 0
06 msg DB 'Vowel count = $'
07
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12     MOV CL, 0
13
14 WHILE_LOOP:
15     MOV AH, 1
16     INT 21H
17     MOV AL, AL
18     CMP AL, '.'
19     JE END_LOOP
20     JE INC_COUNT
21     CMP AL, 'a'
22     JE INC_COUNT
23     CMP AL, 'e'
24     JE INC_COUNT
25     CMP AL, 'i'
26     JE INC_COUNT
27     CMP AL, 'o'
28     JE INC_COUNT
29     CMP AL, 'u'
30     JE INC_COUNT
31     JMP WHILE_LOOP
32 INC_COUNT:
33     INC CL
34     JMP WHILE_LOOP
35 END_LOOP:
36     MOV AH, 9
37     LEA DX, msg
38     INT 21H
39     ADD CL, 30H
40     MOV DL, CL
41     MOV AH, 2
42     INT 21H
43
44 ; Exit
45     MOV AH, 4CH
46     INT 21H
47
48 MAIN ENDP
49 END MAIN
```

emulator screen (80x25 chars)

quertyae-.Vowel count = 3

3. Write a FOR-loop based 8086 program that takes input N and prints '#' N times.

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msg DB 'Enter N: $'
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     MOV AH, 9
13     LEA DX, msg
14     INT 21H
15
16     MOV AH, 1
17     INT 21H
18     SUB AL, 30H
19     MOV CL, AL
20
21 FOR_LOOP:
22     CMP CL, 0
23     JE EXIT
24
25     MOV AH, 2
26     MOV DL, '#'
27     INT 21H
28     DEC CL
29     JMP FOR_LOOP
30
31 EXIT:
32     MOV AH, 4CH
33     INT 21H
34
35 MAIN ENDP
36 END MAIN

```

emulator screen (80x25 chars)

Enter N: 7#####

clear screen change font 8x16

ES 0700 screen source reset aux vars debug stack flags

Tasks for G_Today2.pdf

1. Write an assembly program that reads a single character and classifies it as: - Uppercase letter Lowercase letter - Digit - Special character Use compound AND/OR conditions and display the appropriate category.

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msgU DB 'Uppercase Letter$'
06 msgL DB 'Lowercase Letter$'
07 msgD DB 'Digit$'
08 msgS DB 'Special Character$'
09
10 .CODE
11 MAIN PROC
12     MOV AX, @DATA
13     MOV DS, AX
14     MOV AH, 1
15     INT 21H
16     MOV BL, AL
17     CMP BL, 'A'
18     JB CHECK_LOWER
19     CMP BL, 'Z'
20     JA CHECK_LOWER
21     JMP UPPERCASE
22
23 CHECK_LOWER:
24     CMP BL, 'a'
25     JB CHECK_DIGIT
26     CMP BL, 'z'
27     JA CHECK_DIGIT
28     JMP LOWERCASE
29
30 CHECK_DIGIT:
31     CMP BL, '0'
32     JB SPECIAL
33     CMP BL, '9'
34     JA SPECIAL
35     JMP DIGIT
36
37 UPPERCASE:
38     MOV AH, 9
39     LEA DX, msgU
40     INT 21H
41     JMP EXIT
42
43 LOWERCASE:
44     MOV AH, 9
45     LEA DX, msgL
46     INT 21H
47     JMP EXIT
48
49 DIGIT:
50     MOV AH, 9
51     LEA DX, msgD
52     INT 21H
53     JMP EXIT
54
55 SPECIAL:
56     MOV AH, 9
57     LEA DX, msgS
58     INT 21H
59     JMP EXIT
60
61 EXIT:
62     MOV AH, 4CH
63     INT 21H
64
65 MAIN ENDP
66 END MAIN

```

emulator screen (80x25 chars)

Special Character

clear screen change font 8x16

screen source reset aux vars debug stack flags

2. Write an assembly program that takes input for number of rows and columns and prints a rectangle of '*' characters. - Use nested FOR loops implemented using LOOP instruction - Use JCXZ to prevent unwanted looping when count is zero

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msgR DB 'Enter rows: $'
06 msgC DB 0DH,0AH,'Enter columns: $'
07
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12     MOV AH, 9
13     LEA DX, msgR
14     INT 21H
15
16     MOV AH, 1
17     INT 21H
18     SUB AL, 30H
19     MOV BH, AL
20     MOV AH, 9
21     LEA DX, msgC
22     INT 21H
23
24     MOV AH, 1
25     INT 21H
26     SUB AL, 30H
27     MOV BL, AL
28
29     MOV AL, BH
30     OR AL, AL
31     JZ EXIT
32
33 ROW_LOOP:
34     MOV CL, BL
35     MOV CH, 0
36     JCXZ NEXT_ROW
37
38 COL_LOOP:
39     MOV AH, 2
40     MOV DL, '*'
41     INT 21H
42     LOOP COL_LOOP
43
44     MOV AH, 2
45     MOV DL, 0DH
46     INT 21H
47     MOV DL, 0AH
48     INT 21H
49
50 NEXT_ROW:
51     DEC BH
52     JNZ ROW_LOOP
53
54 EXIT:
55     MOV AH, 4CH
56     INT 21H
57
58 MAIN ENDP
59 END MAIN

```

emulator: output.exe

```

Enter rows: 3
Enter columns: 5*****
*****

```

3. Write an assembly program using a WHILE loop to count vowels in a line of text. - Read characters until a carriage return (0Dh) - Count both uppercase and lowercase vowels - Display the total vowel count at the end

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 msg DB 0DH,0AH,'Total vowels = $'
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11     MOV CX, 0
12
13 WHILE_LOOP:
14     MOV AH, 1
15     INT 21H
16     CMP AL, 0DH
17     JE END_LOOP
18
19     CMP AL, 'a'
20     JE INC_COUNT
21     CMP AL, 'A'
22     JE INC_COUNT
23     CMP AL, 'e'
24     JE INC_COUNT
25     CMP AL, 'E'
26     JE INC_COUNT
27     CMP AL, 'i'
28     JE INC_COUNT
29     CMP AL, 'I'
30     JE INC_COUNT
31     CMP AL, 'o'
32     JE INC_COUNT
33     CMP AL, 'O'
34     JE INC_COUNT
35     CMP AL, 'u'
36     JE INC_COUNT
37     CMP AL, 'U'
38     JE INC_COUNT
39
40     JMP WHILE_LOOP
41
42 INC_COUNT:
43     INC CX
44     JMP WHILE_LOOP
45
46 END_LOOP:
47     MOV AH, 9
48     LEA DX, msg
49     INT 21H
50     ADD CL, 30H
51     MOV DL, CL
52     MOV AH, 2
53     INT 21H
54     MOV AH, 4CH
55     INT 21H
56
57 MAIN ENDP
58 END MAIN

```

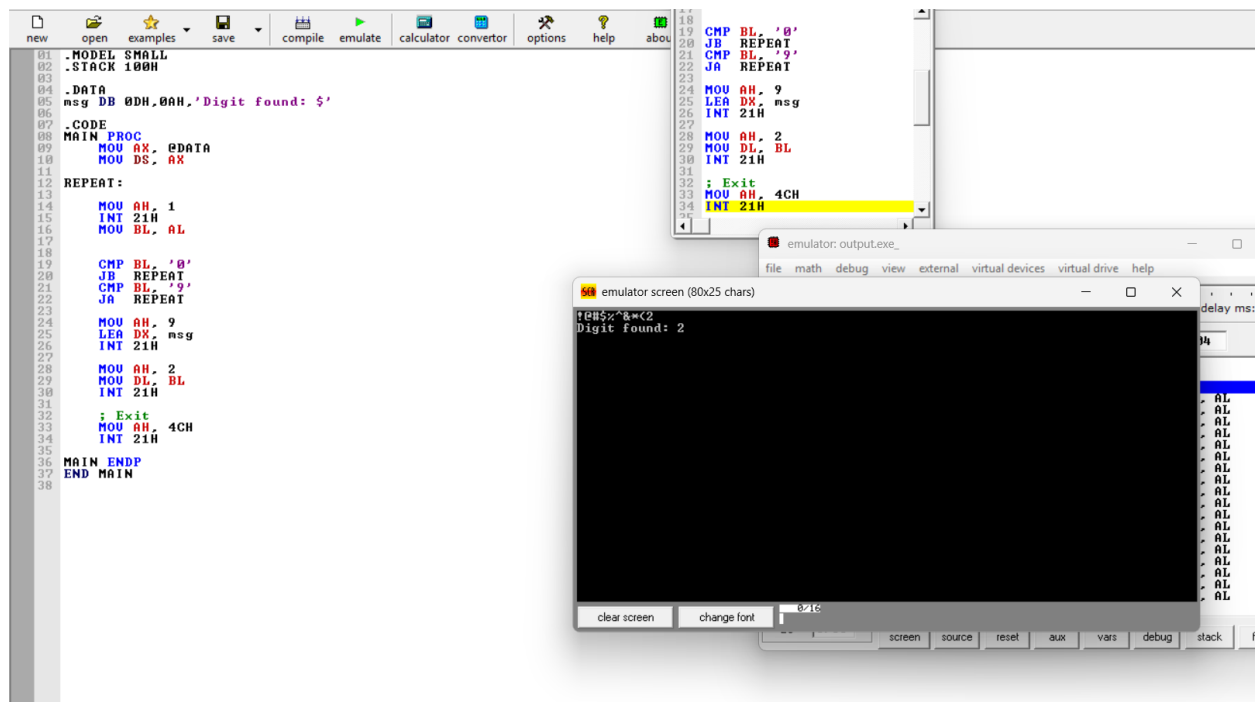
emulator: output.exe

```

aligroh
Total vowels = 3

```

4) Write an assembly program using a REPEAT-UNTIL loop that reads characters until a digit (0-9) is found. - When a digit is detected, display: 'Digit found: X'



5. Write an assembly program to count the number of words in a line. - A word is defined as a sequence of characters separated by spaces - Use WHILE loop with state flags to detect transitions Display total word count at the end

